

Guaranteed Step Accuracy

Combining Agile Decomposition and Test-Driven Development in LLM Software Delivery



Ed Barlow Web Cloud Studio July 2026

Guaranteed Step Accuracy

Ed Barlow — Web Cloud Studio

Abstract

LLM code generation fails multiplicatively: a long build is a chain of steps, and unverified error in any step propagates into every step that consumes its output. This paper describes the quality layer of Drydock, an open specification-driven delivery methodology, which combines the two most durable quality processes in software engineering — Agile decomposition and test-driven development — into a single mechanism for LLM delivery. Agile decomposition reduces an Epic to features and stories, each carrying acceptance criteria authored before any code exists. Test-driven development inverts the build: the acceptance criteria are the story's Definition of Done, declared in the specification, human-owned, and executed after each build step as deterministic, non-agentic checks that consume no model context and cannot be self-reported by the model under evaluation. A story becomes verified only when its programmatic acceptance passes; verification gates the dependency graph, so error cannot propagate past a checked boundary. Each step is therefore individually accurate by construction, and chain accuracy no longer decays with build length. A practical side effect follows: because every step is small, context-bounded, and externally verified, the required model capability per step drops, and smaller, cheaper models complete builds that would otherwise demand frontier models to survive unverified long-horizon execution.

Keywords: test-driven development, acceptance criteria, Agile, LLM code generation, verification, definition of done, step accuracy, model scaling

1. Introduction

Consider a build that requires n sequential steps, each completed correctly with independent probability p . Unverified, the probability the final artifact is correct is p^n : at $p = 0.95$ and $n = 40$ steps, under 13%. This arithmetic — not model quality — is why long agentic builds drift, and why regenerating an application from the same inputs so rarely yields the same working software. Every practitioner has observed the phenomenon; few systems attack its structure.

The structure has two components: steps are too large to verify, and verification, where it exists at all, is performed by the same model that did the work. The software industry solved both problems decades ago, for human developers, with two processes. **Agile** decomposes work into stories small enough to review, each with acceptance criteria agreed before implementation [3]. **Test-driven development** writes the test before the code, so that "done" is an executable fact rather than an author's claim [4]. Drydock's [1] thesis is that these processes transfer to LLM delivery essentially intact — and that their combination is what converts specification-driven development from a promising artifact format into a reproducible engineering discipline. A companion paper [2] treats the delivery-optimization consequences of the same decomposition; this paper treats correctness.

The claim, precisely: **when every atomic story carries acceptance criteria declared before its build and verified deterministically after it, per-step accuracy is guaranteed at each graph boundary, and build correctness becomes independent of build length.** Verified steps reset the error chain; p^n becomes p per step, enforced n times.

2. Agile Decomposition: From Epic to Acceptance-Carrying Stories

Drydock treats the imported specification corpus as an Epic. `drydock analyze` performs Agile decomposition in the model's role as an Agile best-practices team: the Epic splits into features, features into stories, and every story acquires acceptance criteria at

decomposition time — not at build time [1, \$"SAIL Phase 2"]. Three properties of this stage do quality work usually left to hope:

Blockers are first-class. Where the specification is ambiguous or incomplete, analysis emits `BLOCKERS.md` and structured questionnaires rather than guessing. The product owner — Drydock's *Commander* — answers; analysis reruns; the cycle repeats until the plan reports `Ready`. Ambiguity is resolved by a human decision before it can become code. This is the Agile communication loop — refinement, questions, product-owner arbitration — executed between a human and an LLM instead of between developers and a product owner.

Spikes separate research from construction. Genuine unknowns become spike blocks whose product is a recorded `finding:`, so uncertainty is retired once, in writing, rather than re-gambled inside every affected build prompt.

Stories are atomic. Decomposition continues until each story's declared context fits a bounded budget and its behavior is small enough to state acceptance for. A story that cannot be given honest acceptance criteria is, by that fact, not yet decomposed.

The output is a dependency graph of stories — the Manifest — in which every node carries its own verification contract. The graph supplies the *order* of quality: a story may run only when the stories it depends on are `closed/verified`, so no step ever builds atop unverified output [1, \$"Execution Rules"].

3. The TDD Inversion: Acceptance Before Build

Each typed specification file in a Drydock Blueprint terminates in three standard sections: **Programmatic Acceptance**, **User Acceptance**, and **Guardrails** [1, \$"Specification File Format"]. These are authored during planning — before any build step executes — and they are human-owned: the build may add finer-grained tests, but it may never remove or weaken a declared acceptance assertion.

Programmatic Acceptance is the Definition of Done. Each check is a stable heading, an intent statement, and an executable Python assertion. The story and the deterministic tests that prove its acceptance are written in the same build step — the TDD contract, applied at story granularity: the criteria exist first, the build makes them pass.

Verification is deterministic and non-agentic. After each successful story build, the declared checks run as post-build hooks — plain Python invocations, outside any model context. Two consequences matter. First, verification consumes no tokens and cannot be degraded by context pressure. Second, and decisively, *the model cannot self-report success*. An agent asked "did you finish?" exhibits exactly the failure TDD was invented to prevent in humans: sincere, unreliable self-assessment. Drydock never asks. The check passes or the story is `closed/failed`.

Guardrails are permanent negative assertions. Where acceptance criteria assert what the software must do, guardrails assert what it must never do — guarding against model hallucination rather than specification omission. A representative example is Drydock's database encapsulation contract: `DATABASE.md` mandates that all data access flow through a typed class library, and a review finding raw SQL or environment reads outside the encapsulation boundary fails [1, \$"Database Encapsulation"]. Guardrails persist across rebuilds; they are the specification's immune system.

User Acceptance covers the honest remainder. Checks that cannot be truthfully automated — visual quality, workflow feel — are declared as Commander review signals, surfaced with build evidence in the QuarterDeck review console rather than silently skipped or dishonestly mechanized.

4. The Verified State Machine

Every block in the build graph moves through a four-state machine: `pending`, `closed/verified`, `closed/failed`, with a legacy `implemented` state for reconciliation [1, \$"drydock build — PseudoCode State Machine"]. The transitions encode the quality contract:

- A story becomes `closed/verified` only when the build agent succeeds, files are written, *and* programmatic acceptance passes. All three; no override path.
- A story whose build succeeds but whose acceptance fails becomes `closed/failed`, with a single-line failure `finding:` surfaced to the Commander.
- The frontier — the set of runnable blocks — admits only blocks whose external dependencies are `closed/verified`. Failure is therefore *containing*: downstream work does not start on top of a failed foundation.
- `closed/failed` is not terminal. The Commander reopens failed work from the QuarterDeck, revising the block's instructions, acceptance, or scope; the block returns to `pending` with the revision recorded. Recovery is a product-owner decision with an audit trail, never a silent retry.

Every completed block writes evidence to a reviewable per-block file, and material decisions are appended to the Ship's Log, Drydock's append-only decision ledger. The quality system thus produces not only verified software but a verifiable *record*: what was accepted, by which check, on which evidence, decided by whom.

5. Why This Guarantees Step Accuracy

Return to the chain arithmetic of §1. Drydock changes both parameters:

It raises p . Each step is atomic, its context is bounded and complete (the files it implements, compacted interfaces of what it consumes, the Commander's standing intent), and its success condition is stated in the prompt as executable criteria. Small, fully specified, criterion-anchored tasks are the regime where model reliability is highest.

It removes the exponent. Because acceptance runs deterministically at every graph edge, an error cannot cross a verified boundary. A defect is detected at the step that created it, in a context where the fix is local — rerun one story — rather than archaeological. Build correctness degrades from p^n to per-step enforcement: the chain is only ever one unverified step long.

The guarantee is scoped honestly: programmatic acceptance guarantees the software satisfies its *declared* criteria. Criteria quality remains a human responsibility — which is precisely where Agile puts it, with the product owner, at refinement time, supported by an analysis phase whose job is to force the ambiguities out before construction.

6. Side Effect: Smaller Models Suffice

An unplanned but consistent consequence of the architecture: the required model capability per step drops sharply.

Long-horizon agentic coding demands frontier models because the model must *survive* an unverified chain — retaining constraints across a large context, noticing its own errors, and self-correcting without external signal. Drydock externalizes each of those demands into process. Context retention is replaced by deterministic prompt assembly from typed files; error detection is replaced by post-build acceptance checks; recovery is replaced by the state machine and Commander review. What remains for the model is the narrow task frontier models and mid-tier models perform comparably well: implement one bounded story against explicit criteria.

The economics compound with the token accounting of the companion paper [2]: smaller prompts, executed by smaller models, re-verified for free (acceptance is non-agentic and consumes no model context). Teams can reserve frontier models for the phases where judgment density is highest — analysis and planning — and delegate story implementation to cheaper models without surrendering the correctness guarantee, because the guarantee never depended on the model's self-assessment in the first place. Verification effort, not model scale, carries the quality.

7. Related Work

Test-driven development [4] and Agile refinement practice [3] are applied here structurally rather than metaphorically: acceptance-before-build, definition of done, story decomposition, and product-owner arbitration each map to a concrete mechanism (Programmatic Acceptance sections, the verified state machine, Manifest story blocks, the blocker/questionnaire loop). Spec Kit [5] established the specification-and-task-list interface to coding agents; Drydock stories are enriched Spec Kit tasks carrying states, dependencies, and acceptance. Contemporary agent frameworks commonly evaluate completion by model self-report or LLM-as-judge; Drydock's position is that deterministic, non-agentic verification is the only evaluation that composes across a long build. The author is not aware of another published system that combines pre-declared, human-owned acceptance criteria with graph-gated deterministic verification as the delivery mechanism for specification-driven LLM builds.

8. Conclusion

Twenty-five years of Agile and test-driven development taught the industry how humans build software that works: decompose until stories are small enough to state acceptance for, declare done before building, and let executable tests — not the author — decide. Drydock's contribution is the observation that LLMs need this discipline more than humans did, and its implementation: an Epic decomposed into acceptance-carrying atomic stories, a dependency graph that admits no step atop unverified work, and deterministic post-build checks the model can neither see nor game. Step accuracy is enforced at every boundary; correctness stops decaying with build length; and the model needed for each step shrinks to fit the step. The result is the quality property specification-driven development promised but could not deliver alone: working software you can rebuild, and trust, on demand.

References

- [1] E. Barlow. *Drydock Specification: Agile Specification-Driven Design — The SAIL Methodology for Governed Software Delivery*. Web Cloud Studio, 2026. <https://github.com/webcloudstudio/Drydock>
- [2] E. Barlow. *Optimizing Specification-Driven Delivery: Atomic Decomposition, Build Graphs, and Context Engineering for Reproducible LLM Software Delivery*. Web Cloud Studio, 2026.
- [3] K. Beck et al. *Manifesto for Agile Software Development*. 2001. <https://agilemanifesto.org>
- [4] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [5] GitHub. *Spec Kit: Toolkit for Spec-Driven Development*. 2025. <https://github.com/github/spec-kit>